The Fast Fourier Transform

Signals and Linear Systems

Ashrafur Rahman
Lecturer, CSE, BUET

ashrafur@cse.buet.ac.bd

Outline

- **Recap of Last Class**
- **The Need for Speed:** Naive DFT complexity
- **Fast Fourier Transform (FFT):**
  - Radix-2 Decimation-in-Time (DIT)
  - Radix-2 Decimation-in-Frequency (DIF)
- **Conclusion:** FFT Complexity Analysis

Recap of Last Class

- In the last class we derived and looked at the DFT from 2 different perspectives.
  - **Periodic:**
    - DFT can be viewed as the evaluation of the Fourier series coefficients of a *sampled* periodic signal.
    - It can also be viewed as the N unique values of the fourier series of a discrete-time periodic signal.
  - **Aperiodic:**
    - DFT can be viewed as samples of the continuous-time Fourier transform of a *sampled* aperiodic signal.
    - It can also be viewed as samples of the discrete-time fourier transform of a finite-length sequence.
  - As we will see later, if the original conginuous signal is bandlimited, we can recover the original signal from the DFT.

Now let's go into the computational complexity of evaluating the DFT.

- Let's examine the computational complexity of evaluating the DFT directly from its definition:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \qquad \text{where } W_N = e^{-j\frac{2\pi}{N}}$$

- For each of the N values of k, we must compute:
  - N complex multiplications
  - $N - 1$ complex additions
- Therefore, calculating all N values of $X[k]$ explicitly requires $O(N^2)$ operations.
- **Why is this a problem?**
  - Consider 1 second of audio sampled at 44.1 kHz. Here, $N = 44,100$.
  - The naive DFT requires roughly $N^2 \approx 2 \times 10^9$ operations! This would make spectral analysis impossibly slow.

A Brief History of the FFT

- **1805: Carl Friedrich Gauss**
  - Developed the radix-2 algorithm to interpolate asteroid orbits.
  - Predates Joseph Fourier's work on harmonic analysis by over a decade!
  - Left it as a note, unpublished.

Carl Friedrich Gauss

- **1965: James Cooley & John Tukey**
  - Independently rediscovered the algorithm at IBM and Princeton.
  - Their landmark paper launched the modern field of Digital Signal Processing (DSP).
- Many other variants of the FFT have been developed since then. But we are going to study the Cooley-Tukey FFT first.

James W. Cooley

John W. Tukey

- The Cooley-Tukey FFT Algorithm is an elegant $O(N \log N)$ algorithm to compute the DFT by exploiting the symmetry and periodicity of W_N. Recall the DFT definition:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} \qquad \text{where } W_N = e^{-j\frac{2\pi}{N}}$$

- We are first going to study the **radix-2** FFT algorithm which–
  - Successively splits an N-point DFT into two $N/2$-point DFTs.
  - Assumes N is a power of 2 (if not, use zero-padding).
- There are two variants of the Cooley-Tukey FFT algorithm:
  - **Decimation-in-Time (DIT)**
  - **Decimation-in-Frequency (DIF)**

- **Decimation-in-Time** splits the time-domain sequence $x[n]$ into its **even** and **odd** indexed samples:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} = \sum_{r=0}^{N/2-1} x[2r] W_N^{2rk} + \sum_{r=0}^{N/2-1} x[2r+1] W_N^{(2r+1)k}$$

- Since $W_N^{2rk} = e^{-j\frac{2\pi}{N}2rk} = e^{-j\frac{2\pi}{N/2}rk} = W_{N/2}^{rk}$, we get:

$$X[k] = \underbrace{\sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{rk}}_{N/2\text{-point DFT of evens}} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{rk}}_{N/2\text{-point DFT of odds}}$$

- So, we can compute the N-point DFT $X[k]$ by computing two $N/2$-point DFTs!
- But we need some way to convert the $N/2$-point DFTs to N-points.

- Define new functions:

$$G[k] = \sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{rk} = \text{DFT}\{x[2r] \text{ with } N/2 \text{ points}\}$$

$$H[k] = \sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{rk} = \text{DFT}\{x[2r+1] \text{ with } N/2 \text{ points}\}$$

- $G[k]$ and $H[k]$ is clearly defined for $0 \leq k < N/2$.
- But also since $N/2$-point DFT is periodic with period $N/2$, we have:

$$G[k + N/2] = G[k]$$

$$H[k + N/2] = H[k]$$

- So if we copy the first half of $G[k]$ and $H[k]$ to the second half (periodic extension), we can write:

$$X[k] = G[k] + W_N^k H[k]$$

- However, we can do better.

- For the second half, $N/2 \leq k < N$, we can use the property $W_N^{k+N/2} = W_N^k e^{-j\pi} = -W_N^k$:

$$\begin{aligned} X[k+N/2] &= G[k+N/2] + W_N^{k+N/2} H[k+N/2] \\ &= G[k] - W_N^k H[k] \qquad \text{for } N/2 \leq k < N \end{aligned}$$

- So, together we can write:

Radix-2 Decimation-in-Time FFT

For $0 \leq k < N/2$:

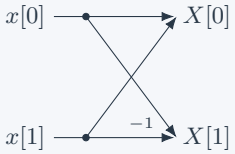
$$\begin{aligned} X[k] &= G[k] + W_N^k H[k] \\ X[k+N/2] &= G[k] - W_N^k H[k] \end{aligned}$$

- This means we only need to compute the $N/2$-point DFTs of the even and odd parts once, and then combine them using simple additions/subtractions and a single complex multiplication (the **"twiddle factor"**) for each output.

Now let us see the computational structure of the FFT by walking through for small values of N.

- The fundamental operation is the 2-point DFT (butterfly).
- Note that $W_2^0 = 1$.

$$X[0] = x[0] + x[1]W_2^0 = x[0] + x[1]$$
$$X[1] = x[0] - x[1]W_2^0 = x[0] - x[1]$$

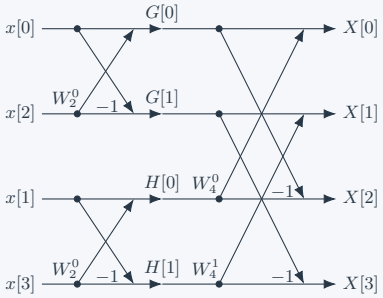


DIT Butterfly Structure ($N = 4$)

- A 4-point DFT is built from two 2-point DFTs.
- $G[k]$ and $H[k]$ (outputs of Stage 1) are combined using twiddle factors W_4^k.

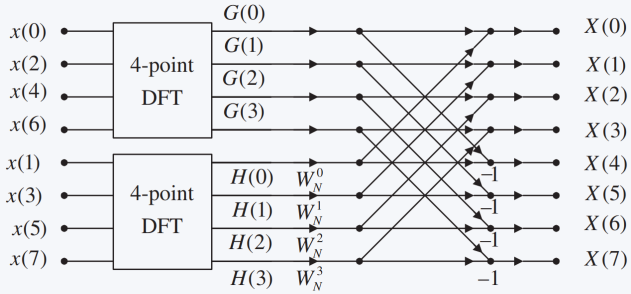
Explicit Formula:

$$X[0] = G[0] + W_4^0 H[0]$$
$$X[1] = G[1] + W_4^1 H[1]$$
$$X[2] = G[0] - W_4^0 H[0]$$
$$X[3] = G[1] - W_4^1 H[1]$$



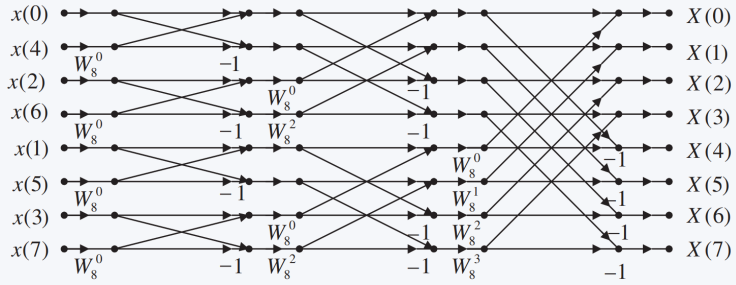
DIT 8-Point FFT: First Iteration

- For $N = 8$, the sequence is split into $N/2 = 4$-point DFTs of even and odd indices.
- The outputs $G(k)$ (evens) and $H(k)$ (odds) are combined using the relation:
- $X[k] = G(k) + W_8^k H(k)$ and $X(k+4) = G(k) - W_8^k H(k)$ for $k = 0, \ldots, 3$.



DIT 8-Point FFT: Full Algorithm

- Expanding all 4-point and 2-point DFTs yields the complete 8-point DIT structure.
- Notice the inputs are in **bit-reversed** order, while outputs are in **natural** order.



Understanding Bit-Reversed Order

- In Radix-2 DIT FFT, the inputs appear in **bit-reversed** order.
- Why? The sequence is successively divided into even and odd indexed halves.
- For $N = 8$, the physical movements happen in three stages:
  1. **Stage 1:** Even (0,2,4,6), Odd (1,3,5,7)
  2. **Stage 2:** Even-Even (0,4), Odd-Even (2,6), Even-Odd (1,5), Odd-Odd (3,7)
  3. **Stage 3:** Single elements: $x[0], x[4], x[2], x[6], x[1], x[5], x[3], x[7]$

Original Index	Binary	Bit-Reversed Binary	New Position
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

- Putting even before odd corresponds to sorting by the rightmost bit.
- Putting even-even before odd-even corresponds to sorting by the second rightmost bit, and so on.
- This is equivalent to sorting the input sequence by the bit-reversed order of their indices.

Iterative Radix-2 DIT FFT

```
Bit-reverse array x of length N (N = 2^v)
for s = 1 to log_2 N:      // For each stage
   M = 2^s      // 2, 4, 8, ..., N
   W_M = e^{-j2π/M}
   for l = 0 to N − M step M:      // M-point DFT blocks in same stage
     W = 1     // Twiddle factors
     for k = 0 to M/2 − 1:      // Butterfly operation
       g = x[l + k]     // G[k]
       h = W · x[l + k + m/2]     // W_M^k H[k]
       x[l + k] = g + h     // X[k]
       x[l + k + m/2] = g − h     // X[k + m/2]
       W = W · W_m
```

Complexity Analysis:

- **Bit-Reversal:** Takes $O(N)$ operations.
  - Swap $x[k]$ with $x[k']$ where k' is the bit-reversal of k.
- **Outer Loop:** Runs $\log_2 N$ times (stages).
- **Inner Loops:** Combined, they iterate over all N elements, performing $N/2$ *butterflies* per stage.
- **Total Complexity:** $\mathbf{O(N \log_2 N)}$ complex multiplications and additions.

We now study the second variant.

- Instead of separating even/odd time indices, **Decimation-in-Frequency** splits the summation $n = 0 \dots N-1$ into the **first half** and **second half** of time indices:

$$\begin{aligned} X[k] &= \sum_{n=0}^{N/2-1} x[n] W_N^{kn} + \sum_{n=N/2}^{N-1} x[n] W_N^{kn} \\ &= \sum_{n=0}^{N/2-1} x[n] W_N^{kn} + \sum_{n=0}^{N/2-1} x[n+N/2] W_N^{k(n+N/2)} \end{aligned}$$

- Since $W_N^{kN/2} = e^{-j\frac{2\pi}{N}k\frac{N}{2}} = e^{-j\pi k} = (-1)^k$, we have:

$$X[k] = \sum_{n=0}^{N/2-1} \Big(x[n] + (-1)^k x[n+N/2] \Big) W_N^{kn}$$

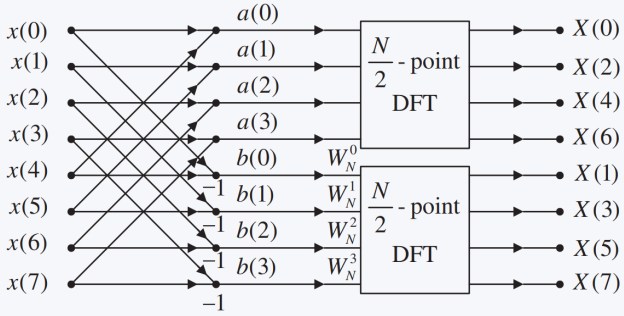
- Now, we decimate in the **frequency** domain by evaluating even and odd k.
- **Even** $k = 2m$: $(-1)^{2m} = 1$, and $W_N^{2mn} = W_{N/2}^{mn}$

$$X[2m] = \sum_{n=0}^{N/2-1} \underbrace{\big(x[n] + x[n+N/2]\big)}_{a(n)} W_{N/2}^{mn} = \mathrm{DFT}_{N/2}\{a(n)\}$$

- **Odd** $k = 2m + 1$: $(-1)^{2m+1} = -1$, and $W_N^{(2m+1)n} = W_N^n W_N^{2mn} = W_N^n W_{N/2}^{mn}$

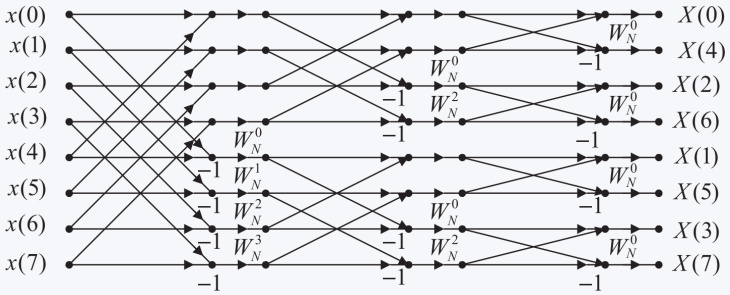
$$X[2m+1] = \sum_{n=0}^{N/2-1} \underbrace{(x[n] - x[n+N/2])\, W_N^n}_{b(n)} W_{N/2}^{mn} = \mathrm{DFT}_{N/2}\{b(n)\}$$

- The fundamental butterfly sequence performs additions *before* the multiplication by the twiddle factor W_N^n:



DIF 8-Point FFT: Full Algorithm

- Expanding the layers gives the full 8-point DIF structure.
- Here inputs are in **natural** order, while outputs emerge in **bit-reversed** order.



The Radix-2 Requirement

Radix-2 FFT algorithms require the sequence length N to be exactly a **power of 2** ($N = 2^m$).

The Problem:

- Real-world data often consists of M samples, where M is **not** a power of 2.
- If we restrict ourselves to M samples, we are stuck with the slow $O(M^2)$ naive DFT!

The Solution: Zero-Padding

- We simply append zeros until we reach the next power of 2: $N = 2^{\lceil \log_2 M \rceil}$.
- **Bonus:** Zero-padding interpolates the spectrum, giving a smoother visualization without altering the true frequency content $X(e^{j\omega})$.
- **However,** if used for periodic $x[n]$, zero-padding changes the period and shape of the periodic signal.
- Can we do FFT without zero-padding?